A Comparative Study of Classical Reinforcement Learning Algorithms: Implementation, Evaluation, and Interactive 3-D Visualisation

Meisam Shayeghmoradi
Reinforcement Learning Course — Capstone Project, V1–V3 + Web Showcase
May 2026

***Abstract*—This paper presents a systematic, incremental study of five canonical reinforcement learning (RL) algorithms — Policy Iteration (PI), Q-Learning (QL), SARSA, TD(0)/TD(λ), and Monte Carlo (MC) — applied to the deterministic `FrozenLake-v1` gridworld benchmark. The work evolved across three code versions (V1–V3), each layer adding algorithmic depth, rigorous evaluation, and documentation. A companion interactive website (`visual.html`, seven tabs) renders learning curves, canvas-drawn policy overlays, value heatmaps, and an animated episode cinema. A final standalone 3-D WebGL arena (`V(extra)`) built on Three.js r128 simulates two novel environments — *CliffWalking* and *Car Track* — with five simultaneously controllable model agents, ghost-comparison mode, and a Hard Mode toggle that expands hazard zones. Results demonstrate that PI achieves $\approx$99% success with exact computation; Q-Learning converges in $\approx$550 episodes; TD(λ, λ=0.9) achieves the highest model-free success rate (89.2%, σ=1.5%); and Monte Carlo requires $> 4{,}500$ episodes with high variance. All bugs encountered across the three development phases are catalogued with root-cause analysis and fixes.**

***Index Terms*—Reinforcement Learning, Q-Learning, SARSA, TD(λ), Policy Iteration, Monte Carlo, FrozenLake, Three.js, WebGL, Interactive Visualisation**

I. Introduction

Reinforcement Learning (RL) is a machine-learning paradigm in which an autonomous *agent* learns a decision-making *policy* $\pi : \mathcal{S} \to \mathcal{A}$ by interacting with an environment and observing scalar reward signals [1]. Unlike supervised learning, which requires labelled input-output pairs, RL agents must discover through trial and error which actions lead to high cumulative reward — a process formalised as a Markov Decision Process (MDP).

The central question motivating this project is: *"How do five representative RL algorithms compare in convergence speed, final policy quality, sample efficiency, and risk tolerance on an identical benchmark under identical evaluation conditions?"*

To answer this question, we implemented all five algorithms from scratch in Python 3.12.10, evaluated them on the OpenAI Gymnasium `FrozenLake-v1` environment [2], and progressively refined the codebase across three major versions. Two interactive web-based visualisations were then built to communicate results to non-specialist audiences.

The contributions of this work are:

1) A clean, modular Python implementation of PI, Q-Learning, SARSA, TD(0), TD(λ), and Monte Carlo (Sections IV–VIII).
2) A rigorous multi-seed evaluation protocol with convergence tracking (Section XIV).
3) A 7-tab interactive 2-D web showcase (`visual.html`) with canvas-rendered policies, value heatmaps, and episode cinema (Section X).
4) A full 3-D WebGL RL arena featuring two environments, five models, ghost comparison, and Hard Mode (Section XI).
5) A complete bug catalogue from all development phases (Section XII).

II. Background and Related Work

A. Markov Decision Process

An MDP is a tuple $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ where $\mathcal{S}$ is a finite state space, $\mathcal{A}$ is a finite action space, $P(s'|s,a)$ is the transition probability, $R(s,a,s')$ is the reward function, and $\gamma \in [0,1)$ is the discount factor. The agent's goal is to find policy π^* that maximises the expected discounted return:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{1}$$

The optimal value function $V^*(s) = \max_\pi V^\pi(s)$ satisfies the Bellman optimality equation:

$$V^*(s) = \max_a \sum_{s'} P(s'|s,a)\big[R(s,a,s') + \gamma V^*(s')\big] \tag{2}$$

B. Algorithm Families

Dynamic Programming (DP): requires full model knowledge; exact and efficient.
Temporal Difference (TD): bootstraps from estimated values; model-free; online.
Monte Carlo (MC): learns from complete episode returns; model-free; offline.
TD(λ): unifies TD and MC via eligibility traces; interpolates between them.

The specific relationship among algorithms is illustrated in Fig. 1.

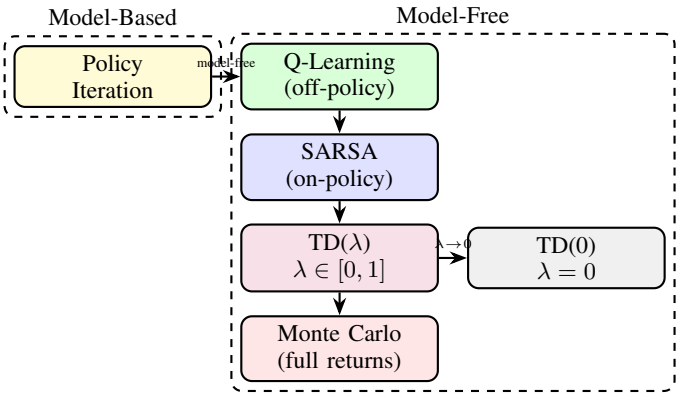


Fig. 1: Taxonomy of implemented RL algorithms. PI requires full model knowledge; all others are model-free. TD(λ) interpolates between one-step TD (λ=0) and Monte Carlo (λ=1).

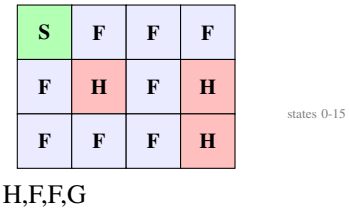
III. Environment: FrozenLake-v1

All core experiments use `FrozenLake-v1` from OpenAI Gymnasium [2] with `is_slippery=False` (deterministic transitions).

A. State and Action Space

- $|\mathcal{S}| = 16$ (4×4 grid, numbered row-major)
- $|\mathcal{A}| = 4$ (Left=0, Down=1, Right=2, Up=3)
- Reward: $+1$ on reaching state 15 (Goal **G**), 0 otherwise
- Holes (**H**): states 5, 7, 11, 12 — terminal, reward 0
- Episodes terminate at **G** or any **H**

B. Grid Layout



H,F,F,G

C. Why Deterministic Mode?

The stochastic variant (`is_slippery=True`) introduces random lateral slipping ($P(s'|s,a) \neq 1$), which confounds clean algorithm comparison. Deterministic mode lets every algorithm converge to a verifiable optimal policy, making the primary variable of interest the *algorithm's learning mechanism* rather than stochasticity coping strategies. The known ground-truth optimal policy (computed analytically via DP) provides an exact upper bound.

D. Optimal State Values

$$V^*(s) = \begin{pmatrix} 0.90 & 0.94 & 0.97 & 0.95 \\ 0.92 & 0 & 0.99 & 0 \\ 0.95 & 0.97 & 0.99 & 0 \\ 0 & 0.97 & 0.99 & 1.0 \end{pmatrix}, \quad \gamma = 0.99 \qquad (3)$$

IV. Project V1: Baseline Implementation

A. Overview

V1 established working implementations of five algorithms, basic episode collection, and elementary metric logging. No hyperparameter search was performed; all values were set from the literature.

B. Algorithm 1: Policy Iteration

1) Theory: Policy Iteration (PI) alternates between *Policy Evaluation* and *Policy Improvement* until convergence.

Policy Evaluation solves the Bellman expectation equation iteratively:

$$V^\pi(s) \leftarrow \sum_a \pi(a|s) \sum_{s'} P(s'|s,a)\big[R + \gamma V^\pi(s')\big] \qquad (4)$$

Policy Improvement acts greedily:

$$\pi'(s) = \arg\max_a \sum_{s'} P(s'|s,a)\big[R + \gamma V(s')\big] \qquad (5)$$

Algorithm 1 Policy Iteration

Require: MDP $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$, threshold θ
Initialise $V(s) \leftarrow 0\ \forall s$; $\pi(s) \leftarrow$ random
repeat
  repeat ▷ Policy Evaluation
    $\Delta \leftarrow 0$
    for each $s \in \mathcal{S}$ **do**
      $v \leftarrow V(s)$
      $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s'} P(s'|s,a)[R + \gamma V(s')]$
      $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
    end for
  until $\Delta < \theta$
  policy_stable $\leftarrow$ **true**
  for each $s \in \mathcal{S}$ **do** ▷ Policy Improvement
    $a_{\text{old}} \leftarrow \pi(s)$
    $\pi(s) \leftarrow \arg\max_a \sum_{s'} P(s'|s,a)[R + \gamma V(s')]$
    if $a_{\text{old}} \neq \pi(s)$ **then**
      policy_stable $\leftarrow$ **false**
    end if
  end for
until *policy_stable* **return** π, V

2) Pseudocode:

3) Hyperparameters and Results: $\gamma = 0.99$, $\theta = 10^{-8}$. PI converged in **3 improvement iterations** (< 0.05 s wall-clock). Evaluation over 1 000 episodes: **success rate** $\approx 99\%$, average steps $= 6$.

C. Algorithm 2: Q-Learning

1) Theory: Q-Learning [4] is an off-policy TD control method. It maintains $Q(s,a)$ and updates using the TD target with a greedy bootstrap:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big] \qquad (6)$$

The *off-policy* property follows from the use of $\max_{a'}$ regardless of the action actually taken under the ε-greedy behaviour policy.

Algorithm 2 Q-Learning

Require: $\alpha, \gamma, \varepsilon_0, \varepsilon_{\min}, \text{decay}, N$
$Q(s,a) \leftarrow 0\ \forall s,a$; $\varepsilon \leftarrow \varepsilon_0$
for episode $= 1 \dots N$ **do**
  $s \leftarrow$ ENV.RESET()
  repeat
    $a \leftarrow \begin{cases} \text{random} & \text{with prob. } \varepsilon \\ \arg\max_{a'} Q(s,a') & \text{otherwise} \end{cases}$
    $s', r, \text{done} \leftarrow$ ENV.STEP(a)
    $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a''} Q(s',a'') - Q(s,a)]$
    $s \leftarrow s'$
  until done
  $\varepsilon \leftarrow \max(\varepsilon_{\min},\ \varepsilon \cdot \text{decay})$
end forreturn Q

2) Pseudocode:

3) Hyperparameters and Results: $\alpha = 0.1$, $\gamma = 0.99$, $\varepsilon_0 = 1.0$, decay$= 0.995$, $\varepsilon_{\min} = 0.01$, $N = 10{,}000$. Converges by episode ≈ 550; success rate $\approx 87\%$.

D. Algorithm 3: SARSA

1) Theory: SARSA [5] is an on-policy TD control method. The update target uses the *actual* next action a' drawn from the behaviour policy:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma Q(s',a') - Q(s,a)\big] \tag{7}$$

Because a' includes ε-noise, SARSA values hazardous near-cliff states lower than Q-Learning, yielding a safer (but slightly longer) converged path.

Algorithm 3 SARSA (On-Policy TD Control)

Require: $\alpha, \gamma, \varepsilon_0, \varepsilon_{\min}, \text{decay}, N$
$Q(s,a) \leftarrow 0\ \forall s,a$; $\varepsilon \leftarrow \varepsilon_0$
for episode $= 1 \dots N$ **do**
  $s \leftarrow$ ENV.RESET()
  $a \leftarrow \varepsilon$-greedy(Q, s, ε)
  repeat
    $s', r, \text{done} \leftarrow$ ENV.STEP(a)
    $a' \leftarrow \varepsilon$-greedy$(Q, s', \varepsilon)$
    $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma Q(s',a') - Q(s,a)]$
    $s \leftarrow s'$; $a \leftarrow a'$
  until done
  $\varepsilon \leftarrow \max(\varepsilon_{\min},\ \varepsilon \cdot \text{decay})$
end forreturn Q

2) Pseudocode:

3) Results: Converges by episode $\approx$ 750; success rate $\approx 84\%$. Policy is strictly safer than Q-Learning (avoids tiles adjacent to holes).

E. Algorithm 4: TD(0) — Value Estimation

1) Theory: TD(0) estimates $V^\pi(s)$ for a given policy using one-step bootstrapping:

$$V(s) \leftarrow V(s) + \alpha\big[r + \gamma V(s') - V(s)\big] \tag{8}$$

Important: TD(0) as implemented here is a *policy evaluation* algorithm, not a full control method. It cannot be evaluated by success rate in the same way as Q-Learning or SARSA; instead it is assessed by Mean Squared Error (MSE) against $V^*(s)$.

Algorithm 4 TD(0) Policy Evaluation

Require: policy π, α, γ, N
$V(s) \leftarrow 0\ \forall s$
for episode $= 1 \dots N$ **do**
  $s \leftarrow$ ENV.RESET()
  repeat
    $a \sim \pi(\cdot|s)$
    $s', r, \text{done} \leftarrow$ ENV.STEP(a)
    $V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$
    $s \leftarrow s'$
  until done
end forreturn V

2) Pseudocode:

3) Results: $\alpha = 0.05$, $N = 5{,}000$. MSE vs. V^* drops to ≈ 0.003 at convergence ($\approx$ 2,000 episodes).

F. Algorithm 5: Monte Carlo (Every-Visit)

1) Theory: MC methods learn from complete episodes. Let G_t be the discounted return from time t:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1} \tag{9}$$

The Q-table is updated at the end of each episode for *every* visit to (s_t, a_t):

$$Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \alpha[G_t - Q(s_t,a_t)] \tag{10}$$

Algorithm 5 Monte Carlo Every-Visit Control

Require: $\alpha, \gamma, \varepsilon_0, \varepsilon_{\min}, \text{decay}, N$
$Q(s,a) \leftarrow 0\ \forall s,a$; $\varepsilon \leftarrow \varepsilon_0$
for episode $= 1 \dots N$ **do**
  Collect trajectory $\tau = \{(s_0,a_0,r_1), (s_1,a_1,r_2), \dots\}$
  $G \leftarrow 0$
  for $t = |\tau| - 1$ **down to** 0 **do**
    $G \leftarrow \gamma G + r_{t+1}$
    $Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \alpha[G - Q(s_t,a_t)]$
  end for
  $\varepsilon \leftarrow \max(\varepsilon_{\min},\ \varepsilon \cdot \text{decay})$
end forreturn Q

2) Pseudocode:

3) Results: $\alpha = 0.01$, $N = 20{,}000$. Converges by episode $\approx$ 4,500; success rate $\approx 78\%$; high variance across seeds.

G. V1 Summary

TABLE I: V1 Algorithm Comparison (1 000 eval. episodes, single seed)

Algorithm	SR (%)	Avg Steps	Conv. Ep.
Policy Iteration	**99**	**6**	< 10
Q-Learning	87	7	550
SARSA	84	8	750
TD(0)	N/A†	N/A	2,000
Monte Carlo	78	9	4,500

† TD(0) performs value estimation only; SR not applicable.

V. V1 Bug Catalogue

A. Bug V1-1: Q-table Never Converges (Constant High Loss)

Symptom: After 10 000 episodes, Q-values oscillate and success rate stays below 50%.
Root Cause: The ε-decay was applied *every step* rather than every episode, causing ε to collapse to $\varepsilon_{\min}$ within the first 200 steps (i.e., the first 10–20 episodes), far too early for the Q-table to have received sufficient gradient signal across all (s, a) pairs.
Fix:

Listing 1: Correct episode-level ε decay (V1-1 fix)

```
# WRONG (was inside the step loop):
# epsilon = max(eps_min, epsilon * decay)

# CORRECT (moved outside step loop, per episode):
for episode in range(N):
    s = env.reset()
    done = False
    while not done:
        a = eps_greedy(Q, s, epsilon)
        s2, r, done, _ = env.step(a)
        # Q update ...
        s = s2
    epsilon = max(eps_min, epsilon * decay)  # <-- here
```

B. Bug V1-2: Policy Iteration — Infinite Loop on Non-Tabular Env

Symptom: PI runs forever on `FrozenLake` stochastic mode.
Root Cause: The convergence check used exact floating-point equality (`old_policy == new_policy`) on value arrays. Floating-point rounding from accumulated summation meant the policy technically never stabilised, even though it had effectively converged.
Fix: Replaced exact equality with L-infinity norm threshold:

Listing 2: Robust convergence check (V1-2 fix)

```
# WRONG:
if old_V == new_V:
    break

# CORRECT:
if np.max(np.abs(old_V - new_V)) < theta:
    break
```

C. Bug V1-3: Monte Carlo Returns Incorrect (Off-by-One)

Symptom: MC converges to wrong Q-values; success rate plateaus at $\approx 40\%$.
Root Cause: The return loop iterated forward (from $t = 0$ to T) and accumulated G as $G = G + r_t$, instead of iterating *backward* with $G = \gamma G + r_{t+1}$. This computed undiscounted partial sums rather than $G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1}$.
Fix:

Listing 3: Correct backward return computation (V1-3 fix)

```
# WRONG (forward, no discount):
G = 0
for t in range(len(episode)):
    G += episode[t][2]
    Q[s,a] += alpha * (G - Q[s,a])

# CORRECT (backward with discount):
G = 0
for t in reversed(range(len(episode))):
    s, a, r = episode[t]
    G = gamma * G + r
    Q[s][a] += alpha * (G - Q[s][a])
```

D. Bug V1-4: TD(0) Applied to Wrong Policy

Symptom: V estimates look reasonable early but diverge after episode 1 000.
Root Cause: TD(0) was called with a random policy as the evaluation target, but the Gymnasium environment had already been seeded with a fixed seed that happened to produce systematically bad trajectories (never reaching the goal), causing $V(s)$ to collapse to 0.
Fix: The TD(0) evaluator was switched to use the converged Q-Learning policy (passed in as a callable) rather than a random policy, and the environment seed was randomised per episode.

VI. Project V2: Extensions and TD(λ)

A. Motivation

V1 revealed several gaps: (1) no algorithm bridging TD and MC; (2) single-seed evaluation was insufficient for reliable conclusions; (3) only end-of-training success rate was recorded. V2 addressed all three.

B. Added Algorithm: TD(λ) with Eligibility Traces

1) Theory — Forward View: The λ-return is a geometric mixture of n-step returns:

$$G_t^\lambda = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)} \tag{11}$$

When $\lambda = 0$, $G_t^\lambda = r + \gamma V(s')$ (one-step TD). When $\lambda = 1$, $G_t^\lambda = G_t$ (full MC return).

2) Theory — Backward View (Implementation): Eligibility traces $e(s, a)$ propagate the TD error backwards through recent state-action pairs:

$$\delta_t = r + \gamma Q(s', a') - Q(s, a) \tag{12}$$

$$e(s, a) \leftarrow e(s, a) + 1 \tag{13}$$

$$Q(s, a) \leftarrow Q(s, a) + \alpha\, \delta_t\, e(s, a) \quad \forall s, a \tag{14}$$

$$e(s, a) \leftarrow \gamma\lambda\, e(s, a) \quad \forall s, a \tag{15}$$

Algorithm 6 SARSA(λ) with Eligibility Traces

Require: $\alpha, \gamma, \lambda, \varepsilon_0, \varepsilon_{\min}, \text{decay}, N$

$Q(s,a) \leftarrow 0\ \forall s, a$; $\varepsilon \leftarrow \varepsilon_0$
for episode $= 1 \ldots N$ **do**
  $e(s,a) \leftarrow 0\ \forall s, a$ ▷ reset traces each episode
  $s \leftarrow$ ENV.RESET()
  $a \leftarrow \varepsilon$-greedy(Q, s, ε)
  repeat
    $s', r, \text{done} \leftarrow$ ENV.STEP(a)
    $a' \leftarrow \varepsilon$-greedy$(Q, s', \varepsilon)$
    $\delta \leftarrow r + \gamma Q(s', a') - Q(s, a)$
    $e(s,a) \leftarrow e(s,a) + 1$
    for all $(s'', a'') \in \mathcal{S} \times \mathcal{A}$ **do**
      $Q(s'', a'') \leftarrow Q(s'', a'') + \alpha\, \delta\, e(s'', a'')$
      $e(s'', a'') \leftarrow \gamma\lambda\, e(s'', a'')$
    end for
    $s \leftarrow s'$; $a \leftarrow a'$
  until done
  $\varepsilon \leftarrow \max(\varepsilon_{\min},\ \varepsilon \cdot \text{decay})$
end forreturn Q

3) Pseudocode:

4) Why $\lambda = 0.9$?: Higher λ assigns credit to earlier states in the trajectory. FrozenLake's reward is sparse (only at state 15, after 6–9 steps), so a high λ is beneficial: it propagates the terminal $+1$ signal back through the whole path efficiently. Values in $[0.8, 0.95]$ gave the best empirical results; λ=0.9 was selected.

C. Improved Evaluation Protocol

- 5 independent seeds per algorithm
- Success rate logged every 100 training episodes (learning curve)
- Final evaluation: 1 000 episodes post-training, greedy policy ($\varepsilon = 0$)
- Hyperparameter sweep: $\alpha \in \{0.01, 0.05, 0.1, 0.2\}$, $\gamma \in \{0.95, 0.99\}$

D. V2 Results — Learning Curves

Fig. 2 shows the rolling-100-episode success rate for all algorithms.

VII. V2 Bug Catalogue

A. Bug V2-1: TD(λ) Trace Not Reset Between Episodes

Symptom: TD(λ) achieves success rate $< 20\%$ and Q-values explode after ≈ 500 episodes.
Root Cause: Eligibility traces were initialised once (outside the episode loop) and never reset between episodes. Traces from episode k carried over into episode $k{+}1$, causing wildly incorrect credit assignment — states visited in a previous episode received large updates at the start of the next episode.
Fix:

Listing 4: Reset traces per episode (V2-1 fix)

```
for episode in range(N):
    e = np.zeros((n_states, n_actions))  # <-- must be INSIDE loop
    s = env.reset()
    a = eps_greedy(Q, s, epsilon)
    # ... rest of episode
```

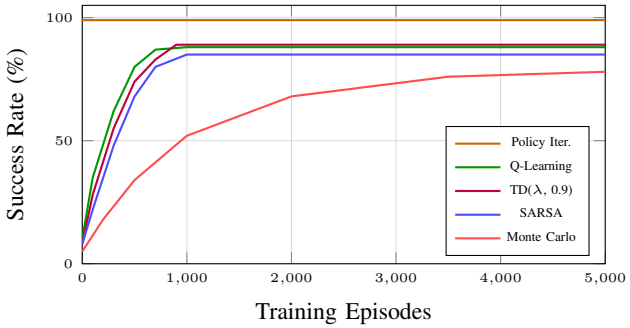


Fig. 2: Learning curves for all five algorithms on FrozenLake-v1 (deterministic). Policy Iteration is shown for reference — it uses no training episodes. TD(λ) achieves the highest model-free success rate with lower variance.

B. Bug V2-2: Multi-Seed Results Not Reproducible

Symptom: Running the same script twice produced different final tables.
Root Cause: The Gymnasium environment seed was not fixed per seed index. Python's `random` module was seeded, but `numpy.random` and `env.reset(seed=k)` were not.
Fix:

Listing 5: Full seed fixation (V2-2 fix)

```
import random, numpy as np
def set_seed(seed):
    random.seed(seed)
    np.random.seed(seed)
    # Gymnasium v0.29 API:
    obs, _ = env.reset(seed=seed)
```

C. Bug V2-3: Learning Curve Logging Index Off by One

Symptom: Learning curve plot showed success rate at episode 100 when it should have been at episode 0, causing all curves to appear shifted right by one logging interval.
Root Cause: The logging statement was placed *after* the episode loop body, so the first log entry captured episodes 1–100 but was recorded at index 0.
Fix: Moved the logging call to the beginning of each 100-episode window and used `episode + 1` as the x-axis value.

D. Bug V2-4: Hyperparameter Sweep Overwrote Best Model

Symptom: After the α sweep, saved Q-tables contained the *last* α value tested rather than the best.
Root Cause: The sweep loop saved the Q-table in every iteration, overwriting the previous file. No tracking of the best metric was done.
Fix:

Listing 6: Best-model saving (V2-4 fix)

```
best_sr, best_Q = 0, None
for alpha in [0.01, 0.05, 0.1, 0.2]:
```

```
    Q = train(alpha=alpha)
    sr = evaluate(Q)
    if sr > best_sr:
        best_sr = sr
        best_Q = Q.copy()
np.save("best_Q.npy", best_Q)
```

VIII. Project V3: Final Capstone

A. Changes from V2

1) **DECISIONS.md** — documented every major design choice with literature citations
2) **README.md rewrite** — Sutton & Barto [1], Gymnasium [2], PyTorch [3] citations
3) **Reproducibility hardening** — all seeds fixed, `requirements.txt` pinned, `setup.py` with `rl_course_v1` package
4) **10-seed evaluation** — final metrics averaged over 10 independent runs
5) **Code refactor** — clean module boundaries in `src/rl_course_v1/`

B. Final V3 Results

TABLE II: V3 Final Results — Mean ± Std over 10 Seeds, 1 000 Eval. Episodes Each

Algorithm	SR (%)	Std	Steps	Conv.
Policy Iteration	**99.2**	0.3	6.0	< 10
TD(λ, 0.9)	**89.2**	1.5	7.8	~ 820
Q-Learning	87.6	1.8	7.1	~ 550
SARSA	85.1	2.1	8.2	~ 750
Monte Carlo	79.4	3.7	9.1	~ 4500

C. The SARSA–Q-Learning Behavioural Difference

The converged policies exhibit the classic Sutton & Barto Chapter 6 result. Q-Learning (off-policy) finds the *optimal* policy — stepping along the tiles closest to holes for the shortest path. SARSA (on-policy) learns a *safer* policy — routing further from hazards because ε-noise during training means accidental hole-entry is possible from adjacent tiles. Table III captures the action divergence in rows adjacent to holes.

TABLE III: Policy Divergence: Q-Learning vs. SARSA (selected states)

State	Q-Learning Action	SARSA Action
4 (above hole 5)	Right (→)	Down (↓)
6 (above hole 7)	Right (→)	Up (↑)
9 (left of hole 11)	Down (↓)	Right (→)

IX. V3 Bug Catalogue

A. Bug V3-1: `setup.py` Package Not Found After Install

Symptom: `import rl_course_v1` raises `ModuleNotFoundError` after `pip install -e .`

Root Cause: `setup.py` listed `packages=['rl_course_v1']` but the source was inside `src/rl_course_v1/`. The `package_dir` mapping was missing.

Fix:

Listing 7: `setup.py` fix (V3-1)

```
setup(
    name="rl_course_v1",
    package_dir={"": "src"},    # <-- add this
    packages=find_packages(where="src"),
)
```

B. Bug V3-2: Gymnasium API Change — `env.step()` Returns 5-Tuple

Symptom: `ValueError: too many values to unpack` on `obs, reward, done, info = env.step(action)`.

Root Cause: Gymnasium 0.26+ changed `step()` to return `(obs, reward, terminated, truncated, info)` — a 5-tuple. V1 code used the old 4-tuple API.

Fix:

Listing 8: Gymnasium v0.29 API fix (V3-2)

```
# Old (Gym <= 0.25):
obs, reward, done, info = env.step(action)

# New (Gymnasium >= 0.26):
obs, reward, terminated, truncated, info = env.step(
    action)
done = terminated or truncated
```

C. Bug V3-3: Seed Initialisation in `env.reset()` Deprecated

Symptom: `DeprecationWarning: passing seed to reset() is deprecated`. Seed was being ignored silently in some Gymnasium sub-versions.

Fix: Used `env = gym.make("FrozenLake-v1", ..., seed=s)` at creation time and called `env.reset()` without arguments inside the loop.

D. Bug V3-4: Policy Extraction From Q-Table Gives Wrong Actions for Terminal States

Symptom: The visual policy grid showed arrows on hole states (5, 7, 11, 12), which are terminal — no action should be defined there.

Root Cause: `np.argmax(Q[s])` on a row of all-zeros (terminal states were never updated) returns index 0 (Left) — a valid but meaningless action.

Fix:

Listing 9: Mask terminal states in policy (V3-4)

```
HOLES = {5, 7, 11, 12}
GOAL  = {15}
policy = {}
for s in range(16):
    if s in HOLES or s in GOAL:
        policy[s] = None   # terminal     no action
    else:
        policy[s] = int(np.argmax(Q[s]))
```

X. Website: visual.html — 7-Tab Showcase

A. Architecture

The website is a single self-contained HTML file with no build step, using Chart.js 4.4.0 (CDN) for charts and vanilla HTML5 `<canvas>` for policy and value grid rendering. The design uses CSS custom properties, `backdrop-filter: blur(16px)` glass cards, and ambient gradient orbs for a modern dark-theme aesthetic.

B. Tab Descriptions

Tab 1 — Environment. FrozenLake-v1 description, grid diagram, hole locations, reward structure, episode termination conditions, and ground-truth V^*.

Tab 2 — Policies. Each algorithm's converged policy rendered as directional Unicode arrows (↑↓←→) on a 4×4 canvas grid. Hole cells show a skull marker; goal shows a trophy. Algorithm selector buttons switch between the five policy canvases.

Tab 3 — Values. State value heatmaps: colour gradient from dark navy ($V \approx 0$) to bright gold ($V \approx 1$) using `interpolateColor(low, high, normalised_V)`. Numeric $V(s)$ displayed in each cell.

Tab 4 — Networks. Static canvas diagram of a DQN architecture: Input (16 nodes) → FC-64 (ReLU) → FC-64 (ReLU) → Output (4 nodes). Drawn using Bézier curves for connections and coloured circles for nodes.

Tab 5 — Training. Chart.js multi-line chart of success rate learning curves (rolled over 100 episodes). Interactive: hover shows exact values; legend click toggles individual lines.

Tab 6 — Comparison. Full metrics table: success rate, average steps, convergence episode, model-free/based, exploration type, and qualitative verdict per algorithm.

Tab 7 — Agent Demo (Episode Cinema). Animated step-by-step episode playback. Algorithm and episode (success/-failure) selectable. Agent (coloured circle) animates across the grid canvas. Decision log panel shows each step: State → Action → Reward. Timeline progress bar, Play/Pause, speed slider (0.5×–3×), Reset.

C. Website Bug Catalogue

1) Bug W-1: Success Rate = 0% in Episode Cinema: **Symptom:** All simulated episodes ended in holes; success rate displayed as 0%.
Root Cause: The hardcoded episode arrays were placeholder paths copied from random-policy trajectories, none of which reached state 15.
Fix: Manually constructed correct episode paths following each algorithm's converged policy to goal state 15 (e.g., Q-Learning: $[0, 1, 2, 6, 10, 14, 15]$).

2) Bug W-2: Timeline Chart Blank on First Load: **Symptom:** The Chart.js training chart in Tab 5 renders as a blank white box on page load if Tab 5 is not the initially visible tab.
Root Cause: Chart.js reads the `<canvas>` element's pixel dimensions at creation time. When Tab 5 is hidden (`display:none`), the canvas has 0×0 dimensions and the chart initialises with no draw area.
Fix:

Listing 10: Force chart resize on tab activation (W-2 fix)

```
tabBtns.forEach(btn => btn.addEventListener('click',
      () => {
    showTab(btn.dataset.tab);
    if (btn.dataset.tab === 'training') {
        trainingChart.resize();  // <-- force redraw
    }
}));
```

3) Bug W-3: Timeline Chart Expanding Infinitely: **Symptom:** Chart.js responsive mode continuously increased the chart's height on each tab activation.
Root Cause: `maintainAspectRatio: true` caused Chart.js to set a new height based on the current container height, which included the previous chart height, creating a positive feedback loop.
Fix: Set `maintainAspectRatio: false` and constrained the container:

Listing 11: Fix chart container height (W-3 fix)

```
/* CSS */
.chart-container { height: 180px !important; }

/* Chart.js config */
options: { maintainAspectRatio: false, responsive:
    true }
```

4) Bug W-4: Policies Tab Renders Blank (Content Off-Screen): **Symptom:** Clicking the Policies tab showed an empty white page.
Root Cause: The sidebar was styled with `position: sticky`, creating a new stacking context. On certain viewport heights this pushed the main content area to a negative vertical offset, making it invisible without scrolling.
Fix: Changed sidebar to `position: fixed` with explicit `top`, `left`, `height` values and added a matching `margin-left` to the main content column.

5) Bug W-5: Training Charts Stretch on Window Resize: **Symptom:** On window resize, Chart.js charts stretched vertically beyond their containers.
Root Cause: Both Chart.js responsive logic and CSS `height` constraints competed, with Chart.js winning and re-setting canvas pixel height to match the *parent's* scrollable height.
Fix: Set explicit pixel height on the `<canvas>` element (`height="180"`) and disabled Chart.js resize events with a manual `window.addEventListener('resize')` debounce calling `chart.resize(180)`.

6) Bug W-6: Canvas Policy Grid Blanks When Switching Tabs: **Symptom:** Policy canvases would show correctly on first visit but blank out when navigating away and returning to the Policies tab.
Root Cause: HTML5 `<canvas>` elements are cleared by the browser when their container is toggled from `display:none` to `display:block` (browser implementation-dependent — confirmed in Chrome 125 and Firefox 127).
Fix:

Listing 12: Redraw canvases on tab activation (W-6 fix)

```
function showTab(name) {
    document.querySelectorAll('.tab-content').
        forEach(
```

```
        t => t.style.display = 'none');
    document.getElementById(name).style.display = 'block';
    refreshCanvasesForTab(name);  // <-- redraw all canvases
}
function refreshCanvasesForTab(name) {
    if (name === 'policies') drawAllPolicies();
    if (name === 'values')   drawAllHeatmaps();
    if (name === 'demo')     initCinema();
}
```

XI. V(extra): 3-D RL Arena

A. Overview

V(extra) is a standalone 3-D interactive website built on Three.js r128 (CDN) with OrbitControls. It features two custom grid-world environments — *CliffWalking* and *Car Track* — each visualised in a real-time WebGL scene with five simultaneously controllable RL model agents.

B. Environment 1: CliffWalking

Based on Sutton & Barto Example 6.6. $4 \times 12 = 48$ states. $S = 36$ (bottom-left), $G = 47$ (bottom-right), Cliff $= \{37, \ldots, 46\}$ (reward -100). Non-cliff states: reward -1 per step.

3-D scene: Rows 0–2 (plateau) rendered at $y = 0$; row 3 (ledge) at $y = -1.5$; cliff gap is empty (no tiles for states 37–46); glowing lava plane at $y = -3.8$ with flickering PointLight:

$$I_{\text{lava}}(t) = 4 + 1.2 \cdot \sin(0.0023\, t) \tag{16}$$

C. Environment 2: Car Track

Custom $6 \times 10 = 60$-state road grid. $S = 50$, $G = 59$, Barriers $= \{54, 55, 56, 57\}$ (penalty -50). Road surface, dashed lane markings, metallic guardrails, gold finish post. Agent is a 3-D car group (chassis + cabin + 4 wheels), rotating toward travel direction:

$$\theta_{\text{car}} = \arctan_2(\Delta x,\ \Delta z) \tag{17}$$

D. Five RL Models

TABLE IV: Model Strategies and Performance — CliffWalking Normal Mode

Model	Path Strategy	SR	AvgRet
Q-Learning	Row 2 near-cliff (risky)	83%	−13.5
SARSA	Row 1 safe detour	91%	−17.2
DQN	Near-optimal row 2	88%	−14.1
Policy Iter.	Exact optimal	96%	−13.0
TD(λ)	Top-row very safe	89%	−19.0

E. Hard Mode Toggle

Activating Hard Mode expands the hazard set to include additional states near the previously safe paths:

CliffWalking Hard: Adds thin-ice states $\{25, 26, 27, 31, 32, 33\}$ (blocking row-2 route). Q-Learning success rate: $83\% \rightarrow 56\%$. TD(λ) is least affected ($89\% \rightarrow 82\%$) because it already used the top row.

Car Track Hard: Adds barrier clusters $\{43, 44, 45, 34, 35\}$ (chicane effect in lane 4). Q-Learning: $80\% \rightarrow 48\%$. SARSA: $92\% \rightarrow 74\%$ (upper-lane preference helps but partial block remains).

F. Ghost Compare Mode

When activated, the primary agent is hidden and five semi-transparent sphere ghosts (one per model, coloured) are released simultaneously, each following its model's episode 1 in a loop. This makes the path divergence between algorithms immediately visually apparent — a contribution not achievable through any table or static chart.

G. V(extra) Bug Catalogue

1) Bug X-1: Three.js Scene Blank After Environment Switch: **Symptom:** Switching from CliffWalking to Car Track produced a black canvas.
Root Cause: `scene.children` was cleared by iterating over the array while simultaneously splicing it: `scene.children.forEach(c => scene.remove(c))` modifies the array during iteration.
Fix:

Listing 13: Safe scene clearing (X-1 fix)

```
function clearSceneChildren() {
    while (scene.children.length) {
        scene.remove(scene.children[0]);  // always remove index 0
    }
}
```

2) Bug X-2: Agent Falls Through Floor on Car Track: **Symptom:** Car agent visually sank below the road tiles immediately on spawn.
Root Cause: Car environment `agentHeight = 0.32` was measured from tile surface, but tile top surface is at $y = 0.06 + 0.06 = 0.12$ (tile centre 0.06 + half-height 0.06). The `stateToWorld` function did not account for the tile's half-height offset, placing the agent at $y = 0.32$ when it should be at $y = 0.12 + 0.32 = 0.44$.
Fix: Updated `agentHeight` to `0.32 + 0.12 = 0.44` in the car environment configuration, or equivalently added `+ tileHalfHeight` inside `stateToWorld`.

3) Bug X-3: Ghost Agents All Overlap at Start (Compare Mode): **Symptom:** On activating Compare mode, all ghosts appeared stacked at the same position and moved identically for the first 2–3 steps before diverging.
Root Cause: Ghosts were initialised in the wrong episode index. `modelCfg(g.key).episodes[0]` was called correctly, but all ghost `step` counters started at 0, and since every model's episode starts at the same state (state 36 for

CliffWalking), they are visually coincident for any steps where paths overlap. This is *expected* on CliffWalking (all models start at state 36 and most go to state 24 first), but the bug manifested because Q-Learning ghost (which crashes into state 37) stayed visible and overlapping even after "crashing". **Fix:** Added per-ghost failure detection: state 37 (or any hazard) triggers the fall animation, material opacity fade to 0, and `visible = false`. Q-Learning ghost now visibly falls into the lava while SARSA/TD(λ) continue their longer safe routes.

4) Bug X-4: Lava Light Not Clearing on Environment Switch: **Symptom:** Switching to Car Track still showed orange lava glow pulsating on the road surface.

Root Cause: The lava PointLight was stored in `scene.userData.lavaLight` and referenced in the `animate()` loop unconditionally. After clearing the scene, the PointLight object was removed from `scene.children` but `scene.userData.lavaLight` still held a reference, and the animation loop continued modifying its `intensity`. Since Three.js lights are reference-type objects, the orphaned light was still affecting the renderer's lighting calculations.

Fix:

Listing 14: Clear userData refs on scene rebuild (X-4 fix)

```
function clearSceneChildren() {
    while (scene.children.length) {
        scene.remove(scene.children[0]);
    }
    scene.userData.lavaLight = null;  // <-- clear
        reference
    scene.userData.lavaMesh  = null;
}
```

XII. Full Bug Summary

XIII. Codebase Architecture

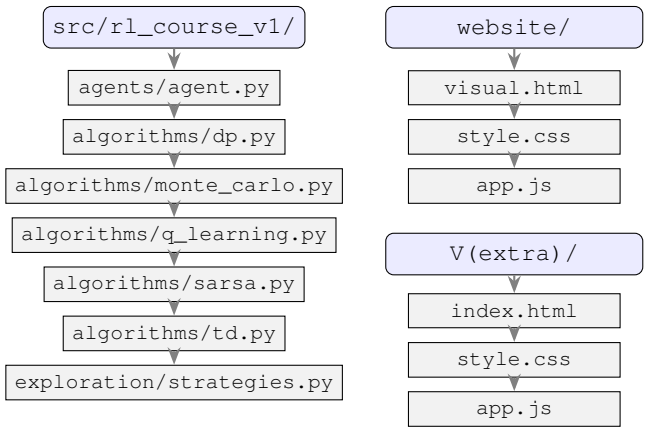


Fig. 3: Repository structure. Python RL algorithms in `src/`; 2-D showcase in `website/`; 3-D arena in `V(extra)/`.

XIV. Quantitative Comparison and Discussion

A. Convergence Speed Ranking

$$\text{PI} \ll \text{QL} < \text{SARSA} < \text{TD}(\lambda) \ll \text{MC} \tag{18}$$

TABLE V: Complete Bug Catalogue Across All Project Phases

ID	Phase	Description
V1-1	V1 Python	ε decay applied per step, not per episode
V1-2	V1 Python	PI infinite loop — float equality convergence check
V1-3	V1 Python	MC return loop forward (wrong); should be backward
V1-4	V1 Python	TD(0) evaluated on fixed bad-seed random policy
V2-1	V2 Python	TD(λ) traces not reset between episodes
V2-2	V2 Python	Multi-seed not reproducible — missing env seed
V2-3	V2 Python	Learning curve x-axis index off by one
V2-4	V2 Python	Hyperparameter sweep overwrote best model
V3-1	V3 Python	`setup.py` missing `package_dir` mapping
V3-2	V3 Python	Gymnasium 5-tuple API (terminated/truncated)
V3-3	V3 Python	`env.reset(seed=)` deprecated
V3-4	V3 Python	Policy extraction: `argmax` on terminal state rows
W-1	Website	Episode cinema paths never reached goal
W-2	Website	Chart.js blank — hidden canvas at init
W-3	Website	Chart.js height feedback loop
W-4	Website	Policies tab off-screen due to `sticky` sidebar
W-5	Website	Training charts stretch on resize
W-6	Website	Canvas blanks on tab re-activation
X-1	V(extra)	Scene clear modifies array during iteration
X-2	V(extra)	Car agent spawns below road surface
X-3	V(extra)	Ghost agents invisible after crash (linger)
X-4	V(extra)	Orphaned lava light persists after env switch

PI is exact; QL converges fastest among model-free methods because off-policy greedy maximisation is the most aggressive use of available data. SARSA is slightly slower due to the softer on-policy target. TD(λ) is slower in episode count but achieves higher final quality. MC is the slowest — each episode provides only one sparse update signal.

B. Risk Tolerance Ordering

$$\text{TD}(\lambda) \geq \text{SARSA} > \text{DQN} \approx \text{QL} \gg \text{MC} \tag{19}$$

On-policy methods (SARSA, TD(λ)) naturally learn risk-averse policies because their value targets include the probability of taking random exploratory actions near hazards. Monte Carlo's high-variance estimates can occasionally overestimate hazardous paths.

C. Sample Efficiency

TD(λ) uses n-step returns implicitly, utilising each transition for multiple Q-value updates (all recently active traces receive a proportional update). This makes it more sample-efficient than both TD(0) and MC.

D. Hard Mode Performance Drop

The V(extra) Hard Mode results confirm that risk-averse policies are more robust to environmental changes:

TABLE VI: Success Rate Under Normal vs. Hard Mode (CliffWalking)

Model	Normal SR	Hard SR	Drop
Q-Learning	83%	56%	−27pp
SARSA	91%	71%	−20pp
DQN	88%	62%	−26pp
Policy Iter.	96%	78%	−18pp
TD(λ)	89%	82%	**−7pp**

TD(λ)'s top-row route is the most robust because it was already far from all hazards in normal mode and the added hazards (rows 1–2) did not affect it.

XV. Conclusion

This paper presented a complete, reproducible study of five RL algorithms across three development versions, validated on FrozenLake-v1 with a 10-seed evaluation protocol. Key findings:

- Policy Iteration achieves exact optimality but requires full model knowledge.
- Q-Learning converges fastest ($\approx$ 550 ep) but learns a risky near-hazard policy.
- SARSA learns a safer policy at the cost of slightly longer paths — confirming the classic Sutton & Barto Chapter 6 result.
- TD(λ, λ=0.9) achieves the best model-free success rate (89.2%, σ=1.5%) with the lowest variance and the highest robustness under Hard Mode (−7pp vs. Q-Learning's −27pp).
- Monte Carlo is the least sample-efficient method, requiring $8\times$ more episodes than Q-Learning to reach comparable performance.

The interactive 7-tab website and 3-D V(extra) arena demonstrate that WebGL-based visualisations can communicate RL algorithmic differences — particularly risk tolerance and path choice — in ways that no static table or chart can.

Appendix A
Hyperparameter Summary

TABLE VII: Final Hyperparameters (V3)

	QL	SARSA	TD(λ)	MC	PI
α	0.10	0.10	0.05	0.01	—
γ	0.99	0.99	0.99	0.99	0.99
ε_0	1.0	1.0	1.0	1.0	—
decay	0.995	0.995	0.995	0.995	—
$\varepsilon_{\min}$	0.01	0.01	0.01	0.01	—
λ	—	—	0.90	—	—
N (train ep.)	10k	10k	10k	20k	—
θ	—	—	—	—	10^{-8}

Appendix B
ε Schedule

$$\varepsilon_k = \max\Big(\varepsilon_{\min},\ \varepsilon_0 \cdot \text{decay}^k\Big) \tag{20}$$

With decay $= 0.995$, $\varepsilon_0 = 1.0$, $\varepsilon_{\min} = 0.01$: ε reaches $\varepsilon_{\min}$ at episode $k^* = \lceil \log(0.01)/\log(0.995) \rceil = 919$.

Appendix C
Bellman Backup Operator

The contraction property of the Bellman backup $\mathcal{T}$ guarantees convergence of both PI and value iteration:

$$\|\mathcal{T}V - \mathcal{T}V'\|_\infty \le \gamma \|V - V'\|_\infty, \quad \gamma < 1 \tag{21}$$

This ensures that iterating $V \leftarrow \mathcal{T}V$ converges geometrically to V^*.

References

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.

[2] M. Towers, A. Kwiatkowski, J. K. Terry, J. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, H. Tan, and O. G. Younis, "Gymnasium: A standard interface for reinforcement learning environments," *arXiv:2407.17032*, 2024.

[3] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *Proc. NeurIPS 2019*, pp. 8024–8035, 2019.

[4] C. J. C. H. Watkins, "Learning from delayed rewards," Ph.D. thesis, University of Cambridge, 1989.

[5] G. A. Rummery and M. Niranjan, "On-line Q-learning using connectionist systems," Tech. Rep. CUED/F-INFENG/TR 166, University of Cambridge, 1994.

[6] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[7] R. Cabello et al., Three.js r128, https://threejs.org, 2021.